RecycleGuard

Evidence-aware decision support
with a human in control

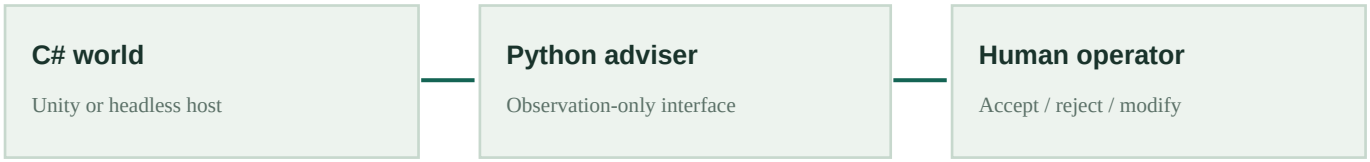
Purpose

A small, inspectable resource-recovery incident simulator explores how explicit constraints and current evidence affect action validity. Operators can accept, reject, modify or request alternatives to advice. The setting and all hazard/resource values are fictional training abstractions.

What is delivered and what is verified

Component	Release status
Shared C# incident model	Compiled; invariants, deterministic replay and resource rules tested.
Python adviser + browser interface	End-to-end tested, including the portable Windows executable.
Unity 3D client	Built with Unity 6000.6.8f1 and run as a Windows player on one machine: automated check, real-click test and recorded walkthrough. Not tested on other hardware or Unity versions.
Model evaluation	Offline fault-injection fixtures evaluated. No real-LLM results.

Architecture: one simulation implementation



Observations -> proposal -> constraints -> approval -> C# revalidation -> state + audit log

Scope of the evidence

1,200 held-out episodes and 5,458 equal-state probes support the behaviour of this synthetic software system. They do not establish real-world safety, improved human decisions or LLM reasoning. The demo has no Deakin/FRV validation or endorsement.

Model, constraints and human authority

Four zones: battery intake, sorting hall, materials store and loading yard. Each has heat, occupants and an isolation flag. Two crews, 18 suppression units and a 32-tick horizon create simple resource constraints.

Transparent discrete dynamics

Heat is an integer from 0 to 100. At heat >=25, powered zones grow by 3-5 units per tick and isolated zones by 1-3. Otherwise they cool by one. An adjacent zone at heat >=75 adds five growth units. Heat >=80 accumulates synthetic damage. Containment requires all occupants cleared and every zone below 25. Damage >=45 or tick 32 terminates an unresolved episode.

Action	Prerequisites and modelled effect
Inspect	Refresh one zone with reconnaissance evidence for three post-action ticks.
Evacuate	Confirmed occupants and one crew; clear occupants; crew reserved until tick+2.
Isolate	Confirmed powered zone; set isolation flag and slow modelled growth.
Suppress	Empty, isolated, actionable zone; one crew and two resource units; heat decreases by 38.
Wait	Advance one decision tick without committing resources.

Invalid current decisions cost one tick but do not consume action resources. Stale revisions and rejected recommendations consume no time. Every executed action is independently validated by C#, including manual overrides.

Observation and policy design

Reliable sensors are current. Delayed sensors lag three ticks after warm-up. Dropout combines intermittent missing zones with one-tick lag elsewhere. Unknown values are explicit (-1); the adviser cannot read hidden simulator truth. Inspection can restore current evidence.

The raw fixture intentionally mixes poor and valid proposals. The permissive checker allows evidence aged up to two ticks, matching the execution guard's observation rule. The strict checker requires age-zero evidence for operational actions and repairs invalid proposals using a transparent priority heuristic. Conditional subgoals are replanned after each action; they are not queued for autonomous execution.

Why the freshness ablation matters

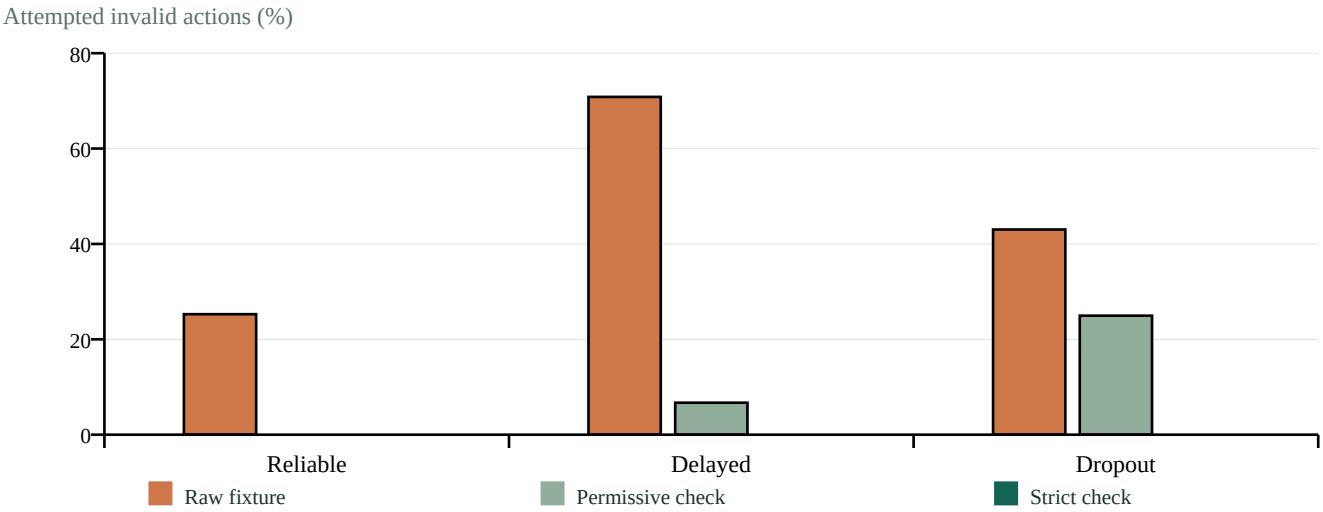
Development tests exposed duplicate evacuation/isolation suggestions from recently aged observations. The world had already changed, while the old observation still met the permissive check. Requiring current evidence removed these failures in the final test set, at the cost of additional inspection decisions.

Human oversight and replay

Advice records the raw proposal, any replacement, violated constraints, observations, timestamps, missing evidence and revision. No confidence percentage is fabricated. Replays rerun all actions and compare stored snapshots before playback. Debrief displays recorded truth, explicitly distinguished from live observations.

Measured results and interpretation

Final evaluation: seeds 2000-2099, three telemetry conditions, four policies, 1,200 episodes. Development seeds 1000-1029 were used to refine the checker and were excluded from the final set. All completed episodes were retained and replay-verified.



Scenario	Raw invalid	Permissive	Strict invalid	Strict ticks
Reliable	25.3%	0.0%	0.0%	6.6
Delayed	70.8%	6.7%	0.0%	14.3
Dropout	43.0%	25.0%	0.0%	15.6

The strict checker contained 100% of these episodes. Raw fixture containment was 100%, 0% and 91% under reliable, delayed and dropout conditions. Under dropout, strict checking took 15.6 ticks on average versus 10.7 for permissive checking. Inspection has a measurable time cost.

Equal-state probes: **5,458** identical observation/proposal pairs; raw invalid proposals **2,964**, strict-check invalid proposals **0**.

Paired effect sizes

Reliable: strict minus raw invalid rate -24.43 percentage points; paired bootstrap 95% interval [-26.10, -22.86].

Delayed: strict minus raw invalid rate -71.40 percentage points; paired bootstrap 95% interval [-72.28, -70.60].

Dropout: strict minus raw invalid rate -40.41 percentage points; paired bootstrap 95% interval [-42.37, -38.21].

Table rates pool decisions; intervals use per-episode paired rates and 2,000 seed-level resamples. Initial conditions are paired, but policy-dependent random draw consumption can diverge. Offline policy timings exclude rendering, C# IPC and model latency.

What these results do not show

Zero invalidity is partly by construction: the checker encodes the model's rules. Fixtures are not LLM error distributions, and the basic script is not an expert baseline. All arms reject invalid execution. No claims about physical fidelity, human trust, real-world safety or AI novelty follow from these numbers.

Verification, reproducibility and provenance

Executed verification

The automated suite contains 52 passing tests covering deterministic replay, the observation-timestamp panel logic, resource invariants, stale requests, unknown evidence, Python/C# constraint parity across generated trajectories, human rejection and alternatives, and real HTTP adapter contracts against a local stub. No live model was used.

Browser automation exercised initial rendering, missing evidence, recommendations, alternatives, rejection without time advance, invalid manual actions, a complete 17-step checked episode, replay export/import/verification, mobile overflow and JavaScript errors. The same flow passed against the portable Windows executable. The browser walkthrough video is browser footage.

Unity client: verified and pending

The client was built in batch mode with Unity 6000.6.8f1 on Windows 10 (GeForce 920M, Direct3D 11). An observation-timestamp panel shows what was observed in each zone, when, and whether it is current, aged, stale or missing; its logic is shared C# tested outside Unity. The player passed its automated check in all three telemetry scenarios, responded correctly to real mouse clicks, and exchanged live advice with the Python service. A captioned walkthrough was recorded from frames the player rendered itself.

The first build passed its automated check while every scene object rendered magenta and labels were unreadable; both were found by inspecting screenshots, fixed, and the check extended. Still pending: reject, alternative and replay-slider controls by click, window resizing, a second machine, and any GitHub workflow run.

Reproduce the evaluation

```
scripts/build-core.ps1
python -m pytest -q
python -m experiments.run --seeds 100 --seed-start 2000
--output artifacts/heldout-evaluation
```

The repository includes per-episode CSV, raw JSONL traces, equal-state probes, source SHA-256 manifests, replay examples, test XML, browser screenshots, build scripts and the complete research protocol. Direct Python dependencies are pinned. The Windows package runs offline without Python or Unity installation; .NET Framework 4.x is required.

Development provenance

The initial implementation and local verification used substantial Codex assistance. Development provenance and contribution guidelines are recorded in docs/CONTRIBUTIONS.md; validation evidence and outstanding checks are tracked in docs/VALIDATION.md.

Next research steps

Evaluate real model outputs on identical cached observations; strengthen scripted/search baselines; introduce sensor noise and delayed command acknowledgements; review scenarios with domain experts; and design an appropriate human study before making claims about decision quality or trust.

Implementation references

Unity command-line builds | UnityWebRequest | JsonUtility
Ollama compatible endpoint | Playwright video recording

Full definitions and limitations: docs/TECHNICAL_REPORT.md and docs/EXPERIMENT_PROTOCOL.md. No FRV data, operational protocols or external endorsements were used.